Photovoltaic Sales Network

Local setup & run instructions · Updated 2026-04-29

Backend — Node.js + Express + MongoDB + Redis (BullMQ workers), TypeScript, Zod, JWT auth

Frontend — React 19 + Vite 8 + TanStack Query + Tailwind CSS v4 + Radix UI

Infra — Docker Compose for local Mongo + Redis

1. Prerequisites

- **Node.js** ≥ 20 — backend uses `tsx` watch mode; frontend targets Vite 8.
- **npm** ≥ 10.
- **Docker** + **Docker Compose** for the bundled Mongo + Redis. (Native installs work too — just point the env vars at them.)
- A modern browser; the SPA is served on `http://localhost:5173`.

2. One-time setup

2.1 Clone & install

```
git clone <repo-url> photovoltaic
cd photovoltaic
```

2.2 Start Mongo + Redis

```
docker compose up -d
```

Mongo binds to `localhost:27017`, Redis to `localhost:6379`. Volumes `mongo_data` and `redis_data` persist between restarts.

2.3 Backend

```
cd backend
cp .env.example .env       # then edit JWT secrets — see env table below
npm install
npm run dev                # API on http://localhost:4000
```

2.4 Frontend

```
cd ../frontend
cp .env.example .env
npm install
npm run dev                # SPA on http://localhost:5173
```

Open `http://localhost:5173/` and sign up. The app runs in dev mode with HMR.

3. Environment variables

3.1 Backend (`backend/.env`)

Variable	Default	Purpose
`NODE_ENV`	`development`	One of `development` / `test` / `production`
`PORT`	`4000`	API listen port
`MONGO_URI`	`mongodb://localhost:27017/photovoltaic`	MongoDB connection string
`REDIS_URL`	`redis://localhost:6379`	Redis connection (used by BullMQ)
`JWT_ACCESS_SECRET`	*required, ≥ 16 chars*	Signing key for short-lived access tokens
`JWT_REFRESH_SECRET`	*required, ≥ 16 chars*	Signing key for refresh tokens
`JWT_ACCESS_TTL`	`15m`	Access-token lifetime (vercel `ms` syntax)
`JWT_REFRESH_TTL`	`7d`	Refresh-token lifetime
`CORS_ORIGIN`	`http://localhost:5173`	Allowed origin for the SPA
`LOG_LEVEL`	`info`	Pino log level (`trace` / `debug` / `info` / `warn` / `error`)

> **Important.** Replace the placeholder `change-me-*` JWT secrets in `.env.example` before booting. The Zod schema in `src/config/env.ts` rejects secrets shorter than 16 characters and exits the process.

3.2 Frontend (`frontend/.env`)

Variable	Default	Purpose
`VITE_API_BASE`	`http://localhost:4000/v1`	Base URL the axios client uses (`src/lib/api.ts`)

Vite reads `.env` at startup — restart `npm run dev` after edits.

4. Scripts

Backend

```
npm run dev          # tsx watch — auto-reload on changes
npm run build        # tsc → dist/
npm start            # node dist/index.js (after build)
npm run typecheck    # tsc --noEmit
npm test             # vitest run (uses mongodb-memory-server — no docker
needed)
npm run test:watch
```

Frontend

```
npm run dev          # Vite dev server :5173
npm run build        # tsc -b && vite build → dist/
npm run preview      # preview the production build
npm run lint         # eslint .
```

5. Background workers

The backend boots BullMQ workers in-process from `src/index.ts`:

- **commission handlers** — emit commissions on contract activation.
- **notification handlers** — fan-out in-app notifications.
- **bonus worker** — monthly bonus job; `scheduleMonthlyBonus()` registers a recurring BullMQ job on Redis.

Workers share the API process today. To split them, move `startBonusWorker()` and `register*Handlers()` into a separate entry point and run two processes against the same Redis.

6. Health check & API

The API exposes `GET /health` returning `{ status: "ok", mongo: "up" }`. All business routes live under `/v1`.

Modules (each at `backend/src/modules/<name>`): `auth`, `users`, `leads`, `customers`, `contracts`, `installations`, `documents`, `catalog` (solutions / bonus rules), `commissions`, `bonuses`, `payments`, `notifications`, `reports`, `territories`, `audit`.

7. Notes & assumptions

- **First-user bootstrap.** No seed script ships with the repo. The first user signs up through the SPA and must be promoted to `ADMIN` manually — update the user's `role` field in the `users` collection (e.g. via Mongo Compass or a one-off `mongosh` command). Once one admin exists, further role assignments can be done in-app.
- **Tests bypass Docker.** Backend tests in `backend/tests/` use `mongodb-memory-server` and spin up an in-process Mongo — no docker compose or local Mongo needed. Redis-dependent code is mocked or not exercised.
- **Workers run in-process.** For local dev there is no separate worker container; all BullMQ workers boot inside the API process. Production deployments should consider splitting them.
- **JWT auth, refresh-on-401.** The frontend's axios interceptor (`frontend/src/lib/api.ts`) automatically calls `POST /v1/auth/refresh` when an access token expires, retries the original request once, and clears auth state if refresh fails. There is a single in-flight refresh promise to avoid stampedes.
- **CORS is single-origin.** `CORS_ORIGIN` accepts one URL. If you serve the SPA from a different origin (e.g. behind a reverse proxy in production), update it accordingly.
- **Tailwind CSS v4.** The frontend uses the v4 plugin model (`@tailwindcss/vite`) — there is no `tailwind.config.{js,ts}`. Theme tokens are declared inline in `frontend/src/index.css` via `@theme {}`.
- **CSV export goes through the API client.** The Reports page downloads CSVs through the configured axios `api` client (`responseType: "blob"`), not raw `fetch` with a relative URL — this is necessary because relative URLs hit the Vite dev server and fall through to `index.html`.
- **Two terminals required for dev.** One for the backend, one for the frontend. (Or use a process manager like `concurrently`; not bundled.)

8. Troubleshooting

Symptom	Likely cause	Fix
Vite "Failed to resolve import …" after `npm install`	Stale Vite dep cache	Stop dev server, `rm -rf frontend/node_modules/.vite`, `npm run dev`
"Export CSV" downloads HTML	Relative URL fetched the SPA fallback page	Already fixed — `downloadCsv` uses the axios `api` client
Backend exits with "Invalid environment configuration"	Missing key or JWT secret < 16 chars	Compare your `.env` to `.env.example`
Mongo / Redis "connection refused"	Containers not running	`docker compose ps` → `docker compose up -d`
SPA blank page, console shows React.Children.only	Old build of `Button.tsx` cached	Hard reload (`Cmd+Shift+R`); the `Slottable` fix is in place

> **Tip.** If you change ports or move the API to a different host, update `VITE_API_BASE` (frontend) *and* `CORS_ORIGIN` (backend) in lockstep. Both must match the actual origins in use.